

101. 对称二叉树

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Easy	(Not Available)	(Not Available)	-	-	0

- **Tags:** 树, 深度优先搜索, 广度优先搜索, 二叉树
- **Companies:** (Not Available)

给你一个二叉树的根节点 root ， 检查它是否轴对称。

示例 1:

输入 : root = [1,2,2,3,4,4,3]
输出 : true

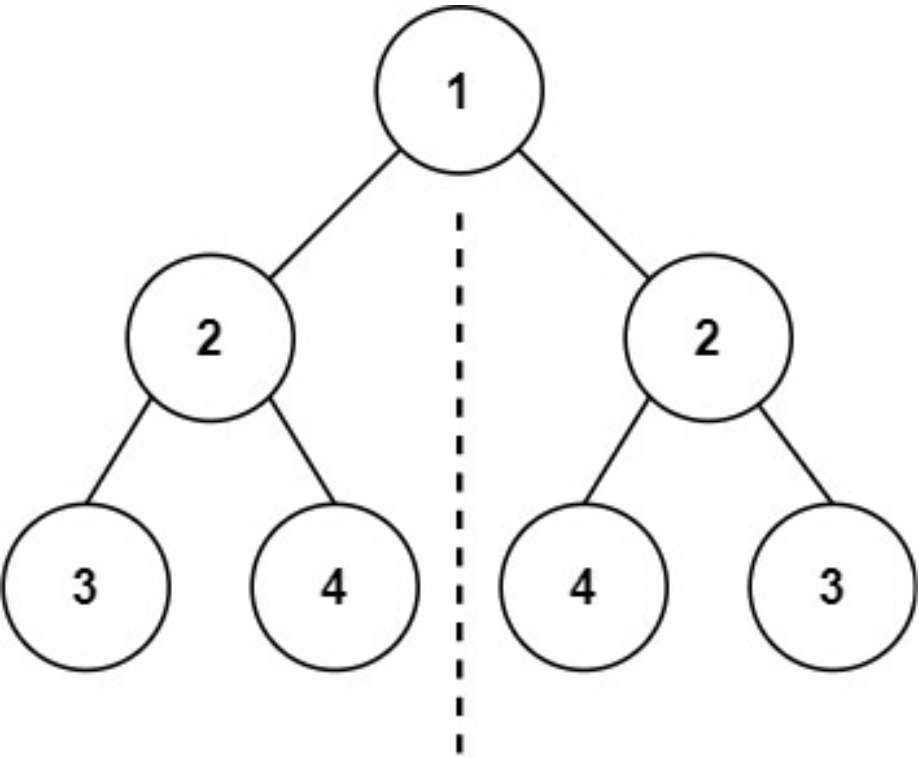


Figure 1: img

示例 2:

输入 : root = [1,2,2,null,3,null,3]

输出：false

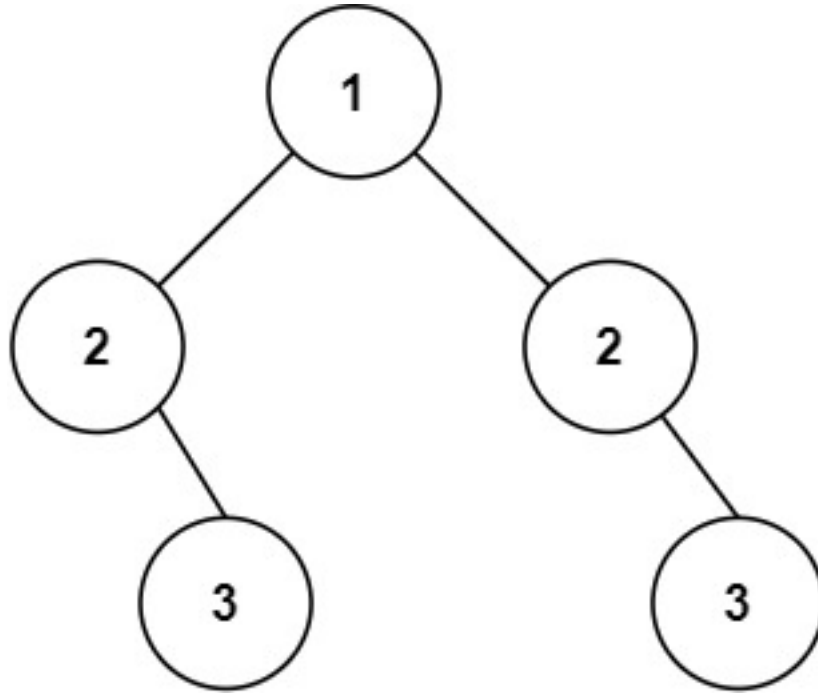


Figure 2: img

提示：

- 树中节点数目在范围 $[1, 1000]$ 内
- $-100 \leq \text{Node.val} \leq 100$

进阶：你可以运用递归和迭代两种方法解决这个问题吗？

Discussion | Solution

解决方法

1. 问题理解

目标：判断一个二叉树是否是轴对称的（镜像对称）。**对称定义：**

- 如果树为空，它是对称的。
- 如果树不为空，则：
 - 根节点的值本身不影响对称性（因为它是对称轴上的点），但其左右子树必须是互相镜像对称的。
 - 两个子树 T1 和 T2 互相镜像对称，当且仅当：
 - T1 的根节点值等于 T2 的根节点值。
 - T1 的左子树与 T2 的右子树镜像对称。
 - T1 的右子树与 T2 的左子树镜像对称。

2. 思路分析

判断整棵树是否对称，核心在于比较其左右子树是否互为镜像。

方法一：递归（深度优先搜索 - DFS） 我们可以定义一个辅助递归函数 `isMirror(node1, node2)`，用于判断以 `node1` 和 `node2` 为根的两个子树是否镜像对称。

1. 基本情况 (Base Cases):

- 如果 `node1` 和 `node2` 都为 `null`，则它们是镜像对称的，返回 `true`。
- 如果 `node1` 和 `node2` 中只有一个为 `null`，或者它们的值不相等 (`node1.val != node2.val`)，则它们不对称，返回 `false`。

2. 递归步骤 (Recursive Step):

- 如果以上基本情况都不满足（即两个节点都存在且值相等），则需要进一步判断它们的子树是否镜像对称。
- 递归判断 `node1` 的左子树 (`node1.left`) 是否与 `node2` 的右子树 (`node2.right`) 镜像对称。
- 递归判断 `node1` 的右子树 (`node1.right`) 是否与 `node2` 的左子树 (`node2.left`) 镜像对称。
- 只有当以上两个递归调用都返回 `true` 时，`node1` 和 `node2` 才构成镜像对称，返回 `true`。否则返回 `false`。

3. 主函数调用:

- 如果根节点 `root` 为 `null`，则视为空树，是对称的，返回 `true`。
- 否则，调用 `isMirror(root.left, root.right)` 来判断根节点的左右子树是否镜像对称。

方法二：迭代（广度优先搜索 - BFS 或深度优先搜索 - DFS 使用栈）

BFS（使用队列） 我们可以使用队列来同时存储并比较需要镜像对称的两个节点。

1. 初始化:

- 如果 `root` 为 `null`，返回 `true`。
- 创建一个队列 `queue`。将根节点的左子节点 `root.left` 和右子节点 `root.right` 成对入队。

2. 遍历队列:

- 当队列不为空时，进行循环：
 - 从队列中成对取出两个节点 `node1` 和 `node2`。
 - 比较 `node1` 和 `node2`:
 - * 如果 `node1` 和 `node2` 都为 `null`，说明这对节点对称，继续下一轮循环 (`continue`)。
 - * 如果 `node1` 和 `node2` 中只有一个为 `null`，或者它们的值不相等 (`node1.val != node2.val`)，则整棵树不对称，直接返回 `false`。
 - 将下一对需要比较的节点入队:
 - * 将 `node1` 的左子节点和 `node2` 的右子节点成对入队 (`queue.append(node1.left), queue.append(node2.right)`)。
 - * 将 `node1` 的右子节点和 `node2` 的左子节点成对入队 (`queue.append(node1.right), queue.append(node2.left)`)。

3. **返回结果**: 如果循环正常结束 (队列为空), 说明所有对称位置的节点都满足条件, 返回 true。

DFS (使用栈) 思路与 BFS 类似, 只是将队列换成栈。

1. **初始化**:
 - 如果 root 为 null, 返回 true。
 - 创建一个栈 stack。将根节点的左子节点 root.left 和右子节点 root.right 成对入栈。
2. **遍历栈**:
 - 当栈不为空时, 进行循环:
 - 从栈中**成对**弹出两个节点 node2 和 node1 (注意出栈顺序与入栈对应)。
 - **比较 node1 和 node2**: (同 BFS)
 - * 都为 null -> continue。
 - * 一个为 null 或值不等 -> return false。
 - **将下一对需要比较的节点入栈**: (顺序与 BFS 相同, 但使用 push 或 append)
 - * stack.append(node1.left)
 - * stack.append(node2.right)
 - * stack.append(node1.right)
 - * stack.append(node2.left)
3. **返回结果**: 如果循环正常结束 (栈为空), 返回 true。

3. 代码实现 (Python)

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

from typing import Optional
import collections

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    # 方法一: 递归 (DFS)
    def isSymmetricRecursive(self, root: Optional[TreeNode]) -> bool:
        """
        使用递归 (DFS) 判断二叉树是否对称
        """
```

```

if not root:
    return True

def isMirror(node1: Optional[TreeNode], node2: Optional[TreeNode]) -> bool:
    # 基本情况 1: 两个都为空, 对称
    if not node1 and not node2:
        return True
    # 基本情况 2: 一个为空, 或值不等, 不对称
    if not node1 or not node2 or node1.val != node2.val:
        return False
    # 递归比较: node1 的左子树 vs node2 的右子树 AND node1 的右子树 vs node2 的左子树
    return isMirror(node1.left, node2.right) and isMirror(node1.right, node2.left)

return isMirror(root.left, root.right)

# 方法二: 迭代 (BFS - 使用队列)
def isSymmetricIterativeBFS(self, root: Optional[TreeNode]) -> bool:
    """
    使用迭代 (BFS) 判断二叉树是否对称
    """
    if not root:
        return True

    queue = collections.deque([root.left, root.right])

    while queue:
        # 成对取出进行比较
        node1 = queue.popleft()
        node2 = queue.popleft()

        # 都为空, 继续下一对比较
        if not node1 and not node2:
            continue
        # 一个为空, 或值不等, 不对称
        if not node1 or not node2 or node1.val != node2.val:
            return False

        # 按镜像对称的要求将下一对节点入队
        # 外侧对: node1.left vs node2.right
        queue.append(node1.left)
        queue.append(node2.right)
        # 内侧对: node1.right vs node2.left
        queue.append(node1.right)
        queue.append(node2.left)

    # 队列为空, 所有对称检查通过

```

```

        return True

# 方法三: 迭代 (DFS - 使用栈)
def isSymmetricIterativeDFS(self, root: Optional[TreeNode]) -> bool:
    """
    使用迭代 (DFS - 栈) 判断二叉树是否对称
    """
    if not root:
        return True

    stack = [root.left, root.right]

    while stack:
        # 成对弹出进行比较 (注意出栈顺序与入栈对应)
        node2 = stack.pop()
        node1 = stack.pop()

        if not node1 and not node2:
            continue
        if not node1 or not node2 or node1.val != node2.val:
            return False

        # 按镜像对称的要求将下一对节点入栈
        # 外侧对: node1.left vs node2.right
        stack.append(node1.left)
        stack.append(node2.right)
        # 内侧对: node1.right vs node2.left
        stack.append(node1.right)
        stack.append(node2.left)

    return True

# 最终提交可以选择其中一种方法
def isSymmetric(self, root: Optional[TreeNode]) -> bool:
    # return self.isSymmetricRecursive(root)
    return self.isSymmetricIterativeBFS(root)
    # return self.isSymmetricIterativeDFS(root)

```

4. 复杂度分析

- 递归方法 (DFS):
 - 时间复杂度: $O(N)$, 其中 N 是树中的节点数。每个节点最多被 `isMirror` 函数访问一次。
 - 空间复杂度: $O(H)$, 其中 H 是树的高度。空间消耗主要来自递归调用栈。最坏情况下 (链状树) 为 $O(N)$, 平均情况下 (平衡树) 为 $O(\log N)$ 。
- 迭代方法 (BFS / DFS):

- 时间复杂度: $O(N)$ 。每个节点最多被入队/入栈和出队/出栈一次。
- 空间复杂度: $O(W)$ 或 $O(H)$ 。
 - * BFS: 空间复杂度取决于队列的最大大小, 即树的最大宽度 W , 最坏情况 $O(N)$ 。
 - * DFS (栈): 空间复杂度取决于栈的最大深度, 即树的高度 H , 最坏情况 $O(N)$ 。

5. 如何在面试中讲解

1. 明确对称定义:

- “首先明确什么是轴对称二叉树: 根节点的左子树和右子树必须互为镜像。”
- “进一步解释镜像对称: 两个节点值相等, 且一个节点的左子树与另一个节点的右子树镜像对称, 一个节点的右子树与另一个节点的左子树镜像对称。”

2. 递归思路 (DFS):

- “最直观的是递归。定义一个辅助函数 `isMirror(node1, node2)` 判断两个节点是否镜像对称。”
- “讲解递归的 Base Cases: 都为空 (`true`), 一个为空或值不等 (`false`)。”
- “讲解递归步骤: 递归比较 `node1.left` 与 `node2.right`, 以及 `node1.right` 与 `node2.left`, 两者都为 `true` 才返回 `true`。”
- “主函数调用 `isMirror(root.left, root.right)`。”
- “分析复杂度: 时间 $O(N)$, 空间 $O(H)$ 。”

3. 迭代思路 (BFS/DFS):

- “为了避免递归栈溢出, 可以使用迭代。核心思想是成对地比较需要对称的节点。”
- “BFS (队列): 将 `root.left` 和 `root.right` 入队。循环中, 成对取出节点比较。如果通过比较, 则按镜像要求将它们的子节点成对入队 (`left1, right2` 和 `right1, left2`)。”
- “DFS (栈): 思路类似 BFS, 只是使用栈, 入栈和出栈顺序需要注意匹配。”
- “讲解迭代中的比较逻辑: 都为空 (`continue`), 一个为空或值不等 (`return false`)。”
- “分析复杂度: 时间 $O(N)$, 空间 BFS 为 $O(W)$, DFS 为 $O(H)$ 。”

4. 对比与选择:

- “递归代码更简洁。迭代方法空间复杂度可能更优 (BFS 在瘦高树上, DFS 在矮胖树上), 且无栈溢出风险。”

5. 边界条件:

- “处理根节点为空的情况。”
- “迭代方法中, 入队/入栈时处理 `null` 节点是关键。”